Wizardry — what to build next

Fifth wave. The first four planned against an instance that did not yet build anything of its own; this one plans against a machine that does. Two commits now run on `wizardry-01` from the fork, the front door is closed, and the last three measurements were made on the live instance rather than on a description of it.

That changes what an honest wave looks like. It is no longer a question of what a chat product could have. It is a question of which three or four things make the *first person who is not you* have a good hour, on an instance with **2 users, 1 guild, 9 channels and 11 messages**.

1. The board, reconciled

Wave three's eighteen rows were written before the build pipeline existed. Six of them have moved, and one of them has changed meaning rather than state:

Row	Said	Now
1.1 Invite-only front door	blocked on owner	**shipped** — `registration.mode = closed`, admin registration URLs on
1.2 Backup that runs without me	next to build	**shipped** — nightly backup, verify clean, one drill run by hand; `rexctl restore` exists
3.1 Manifest splash colour	next to build	**shipped** — `#18271c`, verified through Caddy
3.2 Link card	planned	**shipped** — 1200×630 card, tagline, `summary_large_image`
3.6 Blank screen that says nothing	watchdog planned	**half shipped** — watchdog live and proven; offline page and version handshake still missing
2.2 Message index filled	broken, unfixed	**reclassified** — cold, not proven broken; all nine channels have `indexed_at = NULL` and nothing has ever triggered a backfill

Six of eighteen shipped is worse than it sounds and better than it reads: the six that shipped were the ones that could not be argued with — a lock, a backup, three fields of branding and a sentence that appears when the app fails.

1a. One claim corrected while writing this

The first draft of §3.1 said the web app has no update path at all. A grep for `update available` supported that and a grep is not a finding. `fluxer_app/src/features/platform/types/ Versioning.ts` has seventy-four lines that check for a web build and activate a waiting worker, so the mechanism is real.

What is missing is narrower and more interesting: activating a worker does not re-render the document already on screen. The tab keeps the old JavaScript until someone reloads, which is exactly the stale heading observed on the live instance an hour before this was written. The idea survived the correction and got smaller, which is what a correction is for.

2. The eight questions

The rule for this wave is the rule from every other one: an idea that answers no question someone actually asks is cut before it is written down. Eight survived, and each has a measurement beside it.

#	Question asked out loud	Idea	Evidence	Kind
1	"Will people see what I shipped?"	An update offer: a reload at a moment you choose	Nothing tells the person a new build is waiting. `Versioning.ts` already detects a web update and activates the waiting worker (`SKIP_WAITING` , then waits on `controllerchange`) — but activation does not re-render the document on screen, and no interface asks for the reload. Observed live: the previous build's heading survived a deploy in a tab whose worker had already updated	finding
2	"What happens when the network drops?"	A branded offline page, and a queue for what you typed	No offline document exists anywhere in `fluxer_app` — zero hits for `offline.html` , "you are offline". The worker falls back to a cached page or nothing	finding
3	"Someone new arrived — what did they meet?"	One welcome message on first join	No `sendSystemDm` call in the auth or guild paths; the task type already exists and carries enforcement notices	finding
4	"Can I find what was said?"	Make search true: backfill on first use, and say when the index is still filling	`messages` index 0 documents, last written at creation; all nine channels `indexed_at = NULL` ; indexing is only queued by a search	finding
5	"What does an invite look like before I accept?"	Identity kit: invite splash, guild banner, a mascot emoji pack	The capability ships — `splash_hash` in ten files, `InvitePagePreviewModal` , banner support — and every field is unset on Nightgrove	finding
6	"Does it look like us in my inbox?"	Branded transactional mail	Mail leaves as `noreply@slothlee.xyz` , name `Wizardry` , in upstream's default template; where the bodies are rendered is **unverified**	finding
7	"Is it alive, and is the backup real?"	An operator health page inside the instance	`rexctl doctor` is thirteen checks in a terminal; nothing in the product reports build id, backup age, index counts or registration mode	judgment
8	"Was this fair?"	A moderation decision must carry a written reason	Reports exist (one document in the reports index, an admin surface, 186 audit rows) but nothing requires the reason that later explains it	judgment

Five findings and three judgment calls. The split matters: the findings can be checked in a minute by anyone with the repository, and the judgment calls can be wrong.

3. The ideas, with the rules that keep them honest

3.1 An update offer — the worker is new, the page is not

This idea was written wrong the first time and corrected here (see §1a). The app does not lack an update mechanism: `Versioning.ts` checks for a web build, posts `SKIP_WAITING` to a waiting worker and waits for `controllerchange`. What it lacks is the last step — a person being told that the page in front of them is running code the worker has already replaced, and being offered a reload at a moment that suits them.

- The notice appears once per version, in the corner, and names nothing technical.
- Never reload while something is being typed; the offer waits.
- If either side has no build id, say nothing rather than guess.
- The notice is the difference between "the deploy worked" and "the deploy worked and you are looking at it".

- Never reload while a message is being typed; ask, and let the person choose the moment.
- One notice per version, not one per poll.
- If the build id is missing from either side, say nothing rather than guess.
- This is the only fix that prevents the blank page rather than reporting it.

3.2 An offline page, and a place for what you typed

The service worker already falls back to a cached page; when there is no cached page it falls through to the browser's own error, which is not ours. Ship a self-contained offline document — no fonts, no images, inline everything — with a retry.

- The offline page must not depend on the network for anything, including its typography.
- A message typed offline is held locally and sent when the connection returns, or is marked failed. It is never shown as sent.
- The draft survives a reload; the queue is cleared only after the send returns an id.

3.3 One welcome, on the first join

Not a bot account, and not a personality. The guild says hello once, in plain words, with the three things a new person needs: where to type, what the place is for, and who to ask.

- Sent once per account, keyed on "first guild joined", not on every join.
- No exclamation marks, no "community", British spelling — the voice rule already written down.
- If DMs are closed, say nothing rather than fail silently in a log.
- It must be editable by the owner without a deploy; a message nobody can change is a banner.

3.4 Search that is true

Two halves. The index fills the first time someone searches, and the interface says so while it fills, instead of returning an empty list that reads as "nothing was ever said".

- A search on a cold channel triggers the backfill and shows "indexing this channel" with a count, then results.
- An empty result on a cold index must never render as "no results" — that is a lie with a friendly face.
- Index state belongs in the operator view (§3.7), not in a terminal.

3.5 An identity kit

The largest visible gap between "a chat instance with our colours" and "this place is ours": an invite splash, a guild banner, and a small emoji pack of the mascot. No code — the capability ships and the fields are empty.

- Art is 1920×1080 for the splash, 960×540 for the banner, and carries no text; text in an image is unreadable on the devices that crop it.
- The mascot appears in the emoji pack at 128×128 with the 32 px rule applied: fewer shapes, bigger eyes.
- The invite page must look right with the art missing, because it will be missing until someone uploads it.

3.6 Mail that matches the room

Verification and reset arrive from our domain in someone else's template. The decision already taken stands: the mascot's face in the header, the wordmark in the footer.

- The reset link's host comes from `server.public_url` — the bug that already cost a 404 once.
- Plain-text alternative in every message; a reset that only works in HTML is a reset that fails in a terminal.
- No tracking pixels, no marketing footer: this is a receipt, not a newsletter.

3.7 An operator panel: is it alive

One page, read-only, behind the owner's permissions: build id and commit, when the last backup finished and how large it was, how many documents each search index holds, registration mode, user and message counts, the version of every service.

- Read-only. Nothing on the page changes the instance.
- No secrets, no tokens, no connection strings — counts and times only.
- It reports what it can measure; a value it cannot read shows as unknown, never as zero.

3.8 A decision with a reason

Moderation already records what happened. What it does not require is the sentence that explains it to the person affected.

- A report cannot be closed without a reason, chosen from a short list plus free text.
- The reason is stored with the actor and the timestamp, and is shown to the person affected.
- Nothing here is automatic: no thresholds, no auto-timeouts, no model-written reasons.

4. Order

The build pipeline is proven for `app-proxy` and unproven for everything else. That fact orders the wave more than any product argument:

1. **3.1 + 3.2 together, in one app-proxy build.** Same file tree, same deploy, same revert. These are the last two items of wave three's §3.6 and they close the defect that started it.
2. **3.4, because it is a truth-telling fix.** It also produces the backfill the instance needs.

3. **3.5, because it needs no build at all** — three uploads and one emoji pack, and it is the difference a guest notices first.
4. **3.3, the first change that needs an api image.** Build it, deploy it against a backup, and if the api image has never been proven here, prove it with a no-op change first.
5. **3.6, 3.8, 3.7** after that, in that order: mail is visible, reasons are a rule, and the health page is for you rather than for a guest.

Ideas 1 and 2 are one evening. Ideas 3 to 5 are the ones worth doing before anyone is invited. Ideas 6 to 8 are the ones to do once somebody is actually here.

5. Considered and rejected

- **A bot personality with the mascot's voice.** A second voice in a two-person instance is a costume, not a feature; it also collides with rule 3.3's "editable without a deploy".
- **Paid tiers or premium branding.** Nothing here has a payer, and the fork would have to track upstream's store surface forever.
- **A second search backend.** Meilisearch is running, initialised and healthy; the problem is a cold index, not the engine.
- **A mobile app build.** The desktop and app builds exist upstream; publishing either is a release process, not a feature, and wave four's rule stands — do not claim what is not built.
- **Federation.** It is the most interesting thing this product could do and the least supported by two users and eleven messages.

6. The honest bit

Running count: eighteen rows in wave three, six shipped, and this wave adds eight. That is twenty planned against six shipped, which is the ratio the skill warns about — planning is outrunning building.

So the recommendation is not "build all eight". It is: **build 3.1, 3.2 and 3.4, and then stop planning until a third person has used the instance for a week.** The first two close a defect already observed in the wild. The third makes search honest. Everything after that — welcome, identity, mail, reasons, health — is worth doing *because someone will see it*, and nobody has yet.

What would falsify this wave: after a month, if the instance still has two users and a dozen messages, then ideas 3.3, 3.5 and 3.6 were written for an audience that does not exist. The sample that would make them meaningful is five active members and two hundred messages. Until then, this document's later sections are a shopping list, and it says so rather than pretending otherwise.

7. Decisions that are yours

1. **Who is the second administrator**, if anyone. The board has carried this since wave two.
2. **Whether the instance stays invite-only** once the first guest arrives, or moves to approval — approval means anyone can start an account and you decide afterwards.
3. **Whether 3.3's welcome is a message from the guild or from the mascot**, which is a voice decision and therefore yours.